

HTTP Routing — Lecture Notes

1. What is Routing?

If HTTP **methods** express the *what* (intent/action) of a request, then **routing** expresses the *where* — which resource or address the action should be performed on.

```
GET    /users    → fetch a list of users
POST   /users    → create a new user
```

The server takes the (**method, route**) pair as a composite key and maps it to a specific handler that runs the business logic, performs DB operations, and returns a response.

Routing = mapping URL paths + HTTP methods to server-side handler logic.

2. Types of Routes

2.1 Static Routes

Routes with **no variable parts** — the path string is a constant that never changes.

```
GET    /api/books → returns list of all books
POST   /api/books → creates a new book
```

- Same path `/api/books`, but different methods → two distinct handlers.
- The method + route combination is always unique; they never clash.
- Always returns the same *kind* of response for the same method.

2.2 Dynamic Routes (Path Parameters)

Routes where **part of the path is a variable** supplied by the client — called a **path parameter** or **route parameter**.

```
GET    /api/users/:id
```

- `:id` is a placeholder; the server extracts whatever string appears in that position.
- Convention: `:paramName` — used universally across Node.js, Python, Go, Java, Rust, etc.

- Everything in a route path is treated as a string (even numeric IDs).

Example:

```
GET /api/users/123 → handler extracts id = "123", fetches user 123
GET /api/users/456 → same handler, id = "456", fetches user 456
```

Server-side matching (pseudocode — language-agnostic):

```
r.get("/api/users/:id", handler)
// Matches any GET request of the form /api/users/<anything>
```

When to use: When the route semantically identifies a specific resource instance (e.g., a user, a post, a product).

2.3 Query Parameters

Key-value pairs appended to the URL after a `?`, separated by `&`.

```
GET /api/search?query=some+value
GET /api/books?page=2&limit=20&sort=asc
```

- The server reads these from the query string, not the path.
- **Not part of route matching** — the handler is matched on `/api/search` alone; query params are just data passed to it.

Why not use path params for this?

Putting search/filter values in the path (`/api/search/some-value`) is technically possible but breaks REST semantics — path params are meant to identify *which* resource, not *how* to filter or paginate it.

Common use cases:

Use Case	Example
Search/filter	<code>?query=laptop&brand=dell</code>
Pagination	<code>?page=2&limit=20</code>
Sorting	<code>?sort=price&order=asc</code>
Feature flags	<code>?debug=true</code>

Why GET requests specifically? GET requests have no body in REST. Query params fill that role — they're the way to pass metadata/filters to the server in a GET request.

2.4 Nested Routes

Nesting multiple resource levels in a single path to express hierarchical relationships semantically.

```
/api/users           → all users
/api/users/:userId    → a specific user
/api/users/:userId/posts → all posts of that user
/api/users/:userId/posts/:postId → a specific post of that user
```

Each level of nesting is a separate, valid route with its own handler — they don't clash.

Example from demo:

```
GET /api/users/123/posts/456
```

Reads as: *"Fetch the post with ID 456 belonging to user with ID 123."*

- Each additional segment narrows the scope of the resource.
- Results in different handlers and different response shapes at each level.
- Very common in any API with medium-to-high complexity.

2.5 Route Versioning and Deprecation

Embedding a version identifier in the path prefix to manage breaking changes over time.

```
GET /api/v1/products → returns { id, name, price }
GET /api/v2/products → returns { id, title, price }
```

Why version routes?

- A new client (e.g., mobile app) may need a different response shape than the existing web client.
- Changing the existing route breaks all current consumers.
- Versioning lets both coexist simultaneously.

Deprecation workflow:

1. Release v2 alongside v1.
2. Notify client teams that v1 is deprecated with a migration window.
3. Clients migrate to v2 during that window.
4. Eventually remove v1; rename v2 → v1 if desired.

Versioning avoids the need for ugly route names like `/api/new-products` and provides a clean, stable migration path for all consumers.

2.6 Catch-All Route

A wildcard route defined **last** that matches any request that didn't match any previous handler.

```
GET /api/v3/products → no handler exists → falls through to catch-all
/* → "Route not found" (user-friendly 404 message)
```

- Defined as `/*` or equivalent at the end of all route definitions.
- Without it, unmatched routes return a null/empty response by default — confusing for clients.
- Catch-all returns a meaningful error (typically with status `404`) explaining the route doesn't exist.

3. Summary: Route Types at a Glance

Type	Example	Key Characteristic
Static	<code>GET /api/books</code>	Constant path, no variables
Dynamic (path param)	<code>GET /api/users/:id</code>	Variable segment in path, identifies a specific resource
Query params	<code>GET /api/search?q=foo</code>	Key-value pairs after <code>?</code> , used for filters/pagination
Nested	<code>GET /api/users/:uid/posts/:pid</code>	Hierarchical resource relationships
Versioned	<code>GET /api/v2/products</code>	Breaking changes without disrupting old consumers
Catch-all	<code>/*</code>	Fallback for unmatched routes

4. Path Params vs Query Params — When to Use Which

	Path Parameter	Query Parameter
Purpose	Identify a specific resource	Filter, sort, paginate, or pass metadata

	Path Parameter	Query Parameter
Syntax	<code>/users/:id</code>	<code>/search?q=foo&page=2</code>
Part of route matching?	Yes	No
Used with	GET, PATCH, DELETE on a specific entity	Mostly GET
REST semantics	"Which resource?"	"How to process the resource?"

Quick Revision Checklist

- ☐ Routing = (method + path) → handler mapping
- ☐ Static route: constant path, no variables
- ☐ Dynamic route: `:param` in path, extracted server-side as a string
- ☐ Query params: `?key=val` — used for filters/pagination in GET, not part of route matching
- ☐ Nested routes: hierarchical resource paths, each level is its own valid route
- ☐ Versioning: `/v1/`, `/v2/` prefix — enables breaking changes without disrupting existing consumers
- ☐ Catch-all (`/*`): defined last, handles all unmatched routes with a friendly error
- ☐ Path param = "which resource"; Query param = "how to process it"